

# UDAKE 项目测试规范文档

UDAKE 开发团队

2026 年 3 月 16 日

# 目录

|         |                 |   |
|---------|-----------------|---|
| 1       | 概述与测试策略         | 2 |
| 1.1     | 文档目的            | 2 |
| 1.2     | 项目概述            | 2 |
| 1.3     | 测试策略            | 2 |
| 1.3.1   | 测试分层策略          | 2 |
| 1.3.2   | 测试覆盖率要求         | 3 |
| 1.3.3   | 测试范围            | 3 |
| 1.3.3.1 | 前端测试范围          | 3 |
| 1.3.3.2 | 后端测试范围          | 3 |
| 1.3.3.3 | 功能模块测试范围        | 4 |
| 1.4     | 测试环境            | 4 |
| 1.4.1   | 开发环境            | 4 |
| 1.4.2   | 测试工具            | 4 |
| 2       | 前端测试规范          | 5 |
| 2.1     | 测试框架            | 5 |
| 2.1.1   | Vitest 框架       | 5 |
| 2.1.1.1 | 配置文件            | 5 |
| 2.1.1.2 | 测试脚本            | 6 |
| 2.1.2   | Playwright 框架   | 6 |
| 2.1.2.1 | 配置文件            | 6 |
| 2.2     | 测试文件结构          | 7 |
| 2.2.1   | 测试文件分类          | 7 |
| 2.2.1.1 | API 相关测试（8 个文件） | 7 |
| 2.2.1.2 | 功能模块测试（12 个文件）  | 8 |
| 2.2.1.3 | 性能和安全测试（5 个文件）  | 8 |
| 2.2.1.4 | 服务层测试（3 个文件）    | 8 |
| 2.2.1.5 | UI 和交互测试（4 个文件） | 9 |
| 2.2.1.6 | E2E 测试（1 个文件）   | 9 |

|         |                  |    |
|---------|------------------|----|
| 2.3     | 核心测试用例           | 9  |
| 2.3.1   | API 服务测试         | 9  |
| 2.3.1.1 | 缓存机制测试           | 9  |
| 2.3.1.2 | 错误处理测试           | 10 |
| 2.3.2   | WebSocket 服务测试   | 11 |
| 2.3.3   | 性能监控测试           | 12 |
| 2.3.4   | 安全测试             | 12 |
| 2.3.5   | E2E 测试           | 13 |
| 2.4     | 测试执行             | 14 |
| 2.4.1   | 运行单元测试           | 14 |
| 2.4.2   | 运行 E2E 测试        | 14 |
| 3       | 后端测试规范           | 15 |
| 3.1     | 测试框架             | 15 |
| 3.1.1   | pytest 框架        | 15 |
| 3.1.1.1 | 配置文件             | 15 |
| 3.1.1.2 | 测试依赖             | 15 |
| 3.2     | 测试文件结构           | 16 |
| 3.2.1   | 测试文件分类           | 16 |
| 3.2.1.1 | 核心模块测试（12 个文件）   | 16 |
| 3.2.1.2 | 实时插值系统测试（5 个文件）  | 17 |
| 3.2.1.3 | 多目标优化系统测试（1 个文件） | 17 |
| 3.2.1.4 | API 集成测试（2 个文件）  | 17 |
| 3.3     | 核心测试用例           | 17 |
| 3.3.1   | 模型融合测试           | 17 |
| 3.3.2   | WebSocket 测试     | 18 |
| 3.3.3   | API 测试           | 19 |
| 3.3.4   | 性能测试             | 20 |
| 3.4     | 测试执行             | 21 |
| 3.4.1   | 运行单元测试           | 21 |
| 3.4.2   | 运行异步测试           | 22 |
| 4       | 通讯与集成测试          | 23 |
| 4.1     | 通讯架构             | 23 |
| 4.1.1   | 通讯架构图            | 23 |
| 4.2     | WebSocket 通讯测试   | 23 |
| 4.2.1   | 连接管理测试           | 23 |
| 4.2.2   | 消息处理测试           | 24 |

|          |                       |           |
|----------|-----------------------|-----------|
| 4.2.3    | 任务订阅测试                | 24        |
| 4.3      | HTTP/REST API 通讯测试    | 24        |
| 4.3.1    | 请求处理测试                | 24        |
| 4.3.2    | 响应处理测试                | 24        |
| 4.4      | 集成测试                  | 25        |
| 4.4.1    | 前后端集成测试               | 25        |
| 4.4.2    | WebSocket 与 HTTP 集成测试 | 25        |
| <b>5</b> | <b>功能模块测试</b>         | <b>27</b> |
| 5.1      | 实时插值系统                | 27        |
| 5.1.1    | 功能概述                  | 27        |
| 5.1.2    | 输入输出格式                | 27        |
| 5.1.2.1  | 创建订阅请求                | 27        |
| 5.1.2.2  | 添加数据点请求               | 28        |
| 5.1.2.3  | 查询预测响应                | 28        |
| 5.1.3    | 测试用例                  | 28        |
| 5.1.3.1  | 订阅管理测试                | 28        |
| 5.1.3.2  | 增量更新测试                | 29        |
| 5.1.3.3  | 性能测试                  | 30        |
| 5.2      | 多目标优化系统               | 30        |
| 5.2.1    | 功能概述                  | 30        |
| 5.2.2    | 输入输出格式                | 30        |
| 5.2.2.1  | 优化请求                  | 30        |
| 5.2.2.2  | 优化结果                  | 31        |
| 5.2.3    | 测试用例                  | 32        |
| 5.2.3.1  | 优化算法测试                | 32        |
| 5.2.3.2  | 约束条件测试                | 33        |
| 5.3      | 模型融合系统                | 35        |
| 5.3.1    | 功能概述                  | 35        |
| 5.3.2    | 输入输出格式                | 35        |
| 5.3.2.1  | 创建融合任务请求              | 35        |
| 5.3.2.2  | 融合结果                  | 36        |
| 5.3.3    | 测试用例                  | 36        |
| 5.3.3.1  | 权重计算测试                | 36        |
| 5.3.3.2  | 融合策略测试                | 37        |
| 5.4      | 路径规划系统                | 37        |
| 5.4.1    | 功能概述                  | 37        |
| 5.4.2    | 输入输出格式                | 37        |

|          |                 |           |
|----------|-----------------|-----------|
| 5.4.2.1  | 路径规划请求          | 37        |
| 5.4.2.2  | 路径规划结果          | 38        |
| 5.4.3    | 测试用例            | 39        |
| 5.4.3.1  | 路径计算测试          | 39        |
| 5.4.3.2  | 约束验证测试          | 40        |
| 5.5      | 其他功能模块          | 41        |
| 5.5.1    | 采样建议系统          | 41        |
| 5.5.1.1  | 功能概述            | 41        |
| 5.5.1.2  | 测试用例            | 41        |
| 5.5.2    | 不确定性分级系统        | 41        |
| 5.5.2.1  | 功能概述            | 41        |
| 5.5.2.2  | 测试用例            | 41        |
| <b>6</b> | <b>API 接口测试</b> | <b>43</b> |
| 6.1      | API 接口总览        | 43        |
| 6.1.1    | 接口分类统计          | 43        |
| 6.2      | 核心插值接口          | 43        |
| 6.2.1    | 接口列表            | 43        |
| 6.2.2    | 接口测试用例          | 44        |
| 6.2.2.1  | 启动克里金任务         | 44        |
| 6.2.2.2  | 查询任务状态          | 44        |
| 6.3      | 实时插值系统接口        | 45        |
| 6.3.1    | 接口列表            | 45        |
| 6.3.2    | 接口测试用例          | 46        |
| 6.3.2.1  | 创建订阅            | 46        |
| 6.4      | 多目标优化系统接口       | 47        |
| 6.4.1    | 接口列表            | 47        |
| 6.4.2    | 接口测试用例          | 47        |
| 6.4.2.1  | 创建优化任务          | 47        |
| 6.5      | 模型融合接口          | 48        |
| 6.5.1    | 接口列表            | 48        |
| 6.5.2    | 接口测试用例          | 48        |
| 6.5.2.1  | 创建融合任务          | 48        |
| 6.6      | 路径规划接口          | 49        |
| 6.6.1    | 接口列表            | 49        |
| 6.7      | API 状态码定义       | 50        |

|          |                |           |
|----------|----------------|-----------|
| <b>7</b> | <b>测试文件与用例</b> | <b>51</b> |
| 7.1      | 前端测试文件详细列表     | 51        |
| 7.1.1    | API 相关测试文件     | 51        |
| 7.1.2    | 功能模块测试文件       | 52        |
| 7.1.3    | 性能和安全测试文件      | 52        |
| 7.2      | 后端测试文件详细列表     | 52        |
| 7.2.1    | 核心模块测试文件       | 52        |
| 7.2.2    | 实时插值系统测试文件     | 52        |
| 7.2.3    | 多目标优化系统测试文件    | 52        |
| 7.3      | 测试用例统计         | 52        |
| <b>8</b> | <b>测试执行与报告</b> | <b>54</b> |
| 8.1      | 测试执行流程         | 54        |
| 8.1.1    | 测试准备           | 54        |
| 8.1.2    | 运行测试           | 54        |
| 8.1.2.1  | 前端测试           | 54        |
| 8.1.2.2  | 后端测试           | 55        |
| 8.1.2.3  | API 集成测试       | 55        |
| 8.2      | 测试报告           | 55        |
| 8.2.1    | 覆盖率报告          | 55        |
| 8.2.1.1  | 前端覆盖率报告        | 55        |
| 8.2.1.2  | 后端覆盖率报告        | 56        |
| 8.2.2    | 性能报告           | 56        |
| 8.2.2.1  | 前端性能报告         | 56        |
| 8.2.2.2  | 后端性能报告         | 56        |
| 8.2.3    | 测试结果分析         | 56        |
| 8.3      | 持续集成           | 57        |
| 8.3.1    | CI/CD 配置       | 57        |
| 8.3.2    | 测试策略           | 57        |
| 8.4      | 测试最佳实践         | 57        |
| 8.4.1    | 编写测试用例         | 57        |
| 8.4.2    | Mock 和 Stub    | 57        |
| 8.4.3    | 测试数据管理         | 58        |
| 8.5      | 测试维护           | 58        |
| 8.5.1    | 定期审查           | 58        |
| 8.5.2    | 问题追踪           | 58        |

|                                          |           |
|------------------------------------------|-----------|
| <b>A 数据格式参考</b>                          | <b>59</b> |
| A.1 实时插值数据格式 . . . . .                   | 59        |
| A.1.1 DataPoint 数据结构 . . . . .           | 59        |
| A.1.2 Subscription 数据结构 . . . . .        | 59        |
| A.2 多目标优化数据格式 . . . . .                  | 60        |
| A.2.1 OptimizationRequest 数据结构 . . . . . | 60        |
| A.2.2 ParetoSolution 数据结构 . . . . .      | 60        |
| A.3 模型融合数据格式 . . . . .                   | 60        |
| A.3.1 ModelInput 数据结构 . . . . .          | 60        |
| A.3.2 FusionResult 数据结构 . . . . .        | 61        |
| <b>B 参考文档</b>                            | <b>62</b> |
| B.1 项目文档 . . . . .                       | 62        |
| B.2 API 文档 . . . . .                     | 62        |
| B.3 算法文档 . . . . .                       | 62        |
| B.4 外部资源 . . . . .                       | 62        |
| <b>C 术语表</b>                             | <b>64</b> |

# Chapter 1

## 概述与测试策略

### 1.1 文档目的

本文档旨在为 UDAKE 项目提供全面的测试规范指南，涵盖前端、后端、通讯及功能模块的所有测试相关内容。本文档从测试角度出发，包含测试策略、测试范围、具体测试步骤以及所有功能的输入输出和 API 接口定义。

### 1.2 项目概述

UDAKE 是一个集成了实时插值、多目标优化、模型融合、路径规划等多个专业功能模块的空间数据分析与优化平台。项目采用前后端分离架构，前端使用 TypeScript + React/Vue，后端使用 Python FastAPI，支持 WebSocket 实时通讯。

### 1.3 测试策略

#### 1.3.1 测试分层策略

1. **单元测试**：测试独立的函数、类和组件
2. **集成测试**：测试模块间的交互和数据流
3. **系统测试**：测试完整的业务流程
4. **性能测试**：测试系统性能指标和资源使用
5. **安全测试**：测试安全防护措施和漏洞
6. **E2E 测试**：测试完整的用户 workflows



### 1.3.2 测试覆盖率要求

表 1.1: 测试覆盖率要求

| 覆盖率类型 | 目标值         | 说明             |
|-------|-------------|----------------|
| 语句覆盖率 | $\geq 80\%$ | 至少执行 80% 的代码语句 |
| 函数覆盖率 | $\geq 80\%$ | 至少调用 80% 的函数   |
| 分支覆盖率 | $\geq 80\%$ | 至少覆盖 80% 的条件分支 |
| 行覆盖率  | $\geq 80\%$ | 至少执行 80% 的代码行  |

### 1.3.3 测试范围

#### 1.3.3.1 前端测试范围

- 组件测试：UI 组件的渲染和交互
- 服务层测试：API 服务、WebSocket 服务、配置服务等
- 状态管理测试：数据存储和状态更新
- 性能测试：加载时间、内存使用、Core Web Vitals
- 安全测试：XSS 防护、CSP 策略、敏感信息泄露
- E2E 测试：完整用户流程和跨浏览器测试

#### 1.3.3.2 后端测试范围

- API 测试：所有 API 端点的功能和性能
- 单元测试：核心算法和业务逻辑
- 集成测试：模块间交互和数据流
- 数据库测试：数据存储和查询
- 性能测试：响应时间、吞吐量、资源使用
- WebSocket 测试：实时通讯和消息处理

1.3.3.3 功能模块测试范围

- 实时插值系统：订阅管理、数据点添加、增量更新
- 多目标优化系统：优化算法、约束条件、帕累托解
- 模型融合系统：权重计算、融合策略、性能评估
- 路径规划系统：路径计算、约束验证、优化目标
- 采样建议系统：采样点生成、影响评估、优先级排序
- 不确定性分析：分级算法、风险计算、决策支持

1.4 测试环境

1.4.1 开发环境

- 操作系统：macOS / Linux / Windows
- 前端框架：React/Vue + TypeScript
- 后端框架：Python 3.9+ + FastAPI
- 数据库：SQLite（开发） / PostgreSQL（生产）
- 浏览器：Chrome / Firefox / Safari / Edge

1.4.2 测试工具

表 1.2: 测试工具清单

| 工具类型         | 工具名称                | 用途                         |
|--------------|---------------------|----------------------------|
| 前端单元测试       | Vitest              | JavaScript/TypeScript 单元测试 |
| 前端 E2E 测试    | Playwright          | 端到端测试和跨浏览器测试               |
| 后端单元测试       | pytest              | Python 单元测试和集成测试           |
| 代码覆盖率        | @vitest/coverage-v8 | 代码覆盖率统计                    |
| API 测试       | requests            | HTTP API 测试                |
| 性能测试         | pytest-benchmark    | 性能基准测试                     |
| WebSocket 测试 | pytest-asyncio      | 异步 WebSocket 测试            |

# Chapter 2

## 前端测试规范

### 2.1 测试框架

#### 2.1.1 Vitest 框架

Vitest 是项目使用的主要前端单元测试框架，提供快速的测试执行和现代化的测试 API。

##### 2.1.1.1 配置文件

```
1 import { defineConfig } from 'vitest/config';
2
3 export default defineConfig({
4   test: {
5     environment: 'jsdom',
6     globals: true,
7     coverage: {
8       provider: 'v8',
9       reporter: ['text', 'json', 'html', 'lcov'],
10      exclude: ['node_modules/', 'dist/', 'build/'],
11      thresholds: {
12        lines: 80,
13        functions: 80,
14        branches: 80,
15        statements: 80
16      }
17    }
18  }
19 });
```

Listing 2.1: vitest.config.js

### 2.1.1.2 测试脚本

```
1 {  
2   "scripts": {  
3     "test": "vitest --run",  
4     "test:watch": "vitest",  
5     "test:coverage": "vitest --run --coverage",  
6     "test:security": "vitest --run tests/security.test.ts",  
7     "test:e2e": "playwright test",  
8     "test:e2e:ui": "playwright test --ui",  
9     "test:performance": "vitest --run tests/performance-monitor.test.ts"  
10  }  
11 }
```

Listing 2.2: package.json 测试脚本

## 2.1.2 Playwright 框架

Playwright 用于端到端测试，支持多浏览器和移动设备测试。

### 2.1.2.1 配置文件

```
1 import { defineConfig, devices } from '@playwright/test';  
2  
3 export default defineConfig({  
4   testDir: './tests/e2e',  
5   fullyParallel: true,  
6   retries: process.env.CI ? 2 : 0,  
7   workers: process.env.CI ? 1 : undefined,  
8   reporter: 'html',  
9   use: {  
10    baseURL: 'http://localhost:5173',  
11    trace: 'on-first-retry',  
12  },  
13  projects: [  
14    {  
15      name: 'chromium',  
16      use: { ...devices['Desktop Chrome'] },  
17    },  
18    {  
19      name: 'firefox',  
20      use: { ...devices['Desktop Firefox'] },  
21    },  
22    {
```

```
23     name: 'webkit',
24     use: { ...devices['Desktop Safari'] },
25   },
26   {
27     name: 'Mobile Chrome',
28     use: { ...devices['Pixel 5'] },
29   },
30   {
31     name: 'Mobile Safari',
32     use: { ...devices['iPhone 12'] },
33   },
34 ],
35 webServer: {
36   command: 'npm run dev',
37   url: 'http://localhost:5173',
38   reuseExistingServer: !process.env.CI,
39 },
40 });
```

Listing 2.3: playwright.config.ts

## 2.2 测试文件结构

### 2.2.1 测试文件分类

前端测试文件共 39 个，按功能分类如下：

#### 2.2.1.1 API 相关测试（8 个文件）

- tests/api.test.js - API 基础功能测试
- tests/api/api-service.test.ts - API 服务详细测试
- tests/anomaly-detect-api.test.js - 异常检测 API 测试
- tests/config-api.test.js - 配置 API 测试
- tests/decision-thresholds-api.test.js - 决策阈值 API 测试
- tests/error-predict-api.test.js - 误差预测 API 测试
- tests/model-evaluation-api.test.js - 模型评估 API 测试
- tests/risk-calculate-api.test.js - 风险计算 API 测试

- `tests/risk-report-api.test.js` - 风险报告 API 测试
- `tests/uncertainty-classify-api.test.js` - 不确定性分类 API 测试

#### 2.2.1.2 功能模块测试 (12 个文件)

- `tests/advancedfilter.test.js` - 高级过滤器测试
- `tests/batchoperations.test.js` - 批量操作测试
- `tests/confirmdialog.test.js` - 确认对话框测试
- `tests/datacomparison.test.js` - 数据对比测试
- `tests/exportenhancer.test.js` - 导出增强测试
- `tests/feedbackcollector.test.js` - 反馈收集器测试
- `tests/formvalidator.test.js` - 表单验证器测试
- `tests/historymanager.test.js` - 历史管理器测试
- `tests/offlinemanager.test.js` - 离线管理器测试
- `tests/preferencespanel.test.js` - 偏好设置面板测试
- `tests/templatedownloader.test.js` - 模板下载器测试
- `tests/thememanager.test.js` - 主题管理器测试

#### 2.2.1.3 性能和安全测试 (5 个文件)

- `tests/performance-monitor.test.ts` - 性能监控测试
- `tests/performancemonitor.test.js` - 性能监控器测试
- `tests/cache-performance.test.js` - 缓存性能测试
- `tests/security.test.ts` - 安全测试套件
- `tests/performance-benchmark.js` - 性能基准测试

#### 2.2.1.4 服务层测试 (3 个文件)

- `tests/websocket.test.ts` - WebSocket 服务测试
- `tests/location-service.test.js` - 位置服务测试
- `tests/cache.test.js` - 缓存系统测试

### 2.2.1.5 UI 和交互测试 (4 个文件)

- tests/gesture-enhancement.test.js - 手势增强测试
- tests/keyboardmanager.test.js - 键盘管理器测试
- tests/loadingmanager.test.js - 加载管理器测试
- tests/skeletonloader.test.js - 骨架屏加载器测试

### 2.2.1.6 E2E 测试 (1 个文件)

- tests/e2e/workflow.test.ts - 完整工作流 E2E 测试

## 2.3 核心测试用例

### 2.3.1 API 服务测试

#### 2.3.1.1 缓存机制测试

```
1 describe('缓存机制', () => {
2   it('GET 请求应该被缓存', async () => {
3     mockFetch.mockResolvedValueOnce({
4       ok: true,
5       json: () => Promise.resolve({ data: 'test' })
6     });
7
8     const url = 'http://localhost:8000/api/test';
9     const result1 = await api.request(url);
10    const result2 = await api.request(url);
11
12    expect(mockFetch).toHaveBeenCalledTimes(1);
13    expect(result1).toEqual({ data: 'test' });
14    expect(result2).toEqual({ data: 'test' });
15  });
16
17  it('POST 请求不应该被缓存', async () => {
18    mockFetch.mockResolvedValue({
19      ok: true,
20      json: () => Promise.resolve({ ok: true })
21    });
22
23    const url = 'http://localhost:8000/api/test';
24    await api.request(url, { method: 'POST', body: '{}' });
```

```
25     await api.request(url, { method: 'POST', body: '{}' });
26
27     expect(mockFetch).toHaveBeenCalledTimes(2);
28   });
29
30   it('clearCache 应该清除所有缓存', async () => {
31     mockFetch.mockResolvedValue({
32       ok: true,
33       json: () => Promise.resolve({ data: 1 })
34     });
35
36     await api.request('http://localhost:8000/api/test');
37     api.clearCache();
38     await api.request('http://localhost:8000/api/test');
39
40     expect(mockFetch).toHaveBeenCalledTimes(2);
41   });
42 });
```

Listing 2.4: 缓存机制测试用例

### 2.3.1.2 错误处理测试

```
1 describe('错误处理', () => {
2   it('HTTP 错误应该抛出异常', async () => {
3     mockFetch.mockResolvedValueOnce({
4       ok: false,
5       status: 400,
6       json: () => Promise.resolve({ detail: '服务器错误' })
7     });
8     await expect(api.request(url)).rejects.toThrow('服务器错误');
9   });
10
11   it('网络错误应该重试', async () => {
12     mockFetch.mockRejectedValueOnce(new Error('Network error'));
13     mockFetch.mockRejectedValueOnce(new Error('Network error'));
14     mockFetch.mockResolvedValueOnce({
15       ok: true,
16       json: () => Promise.resolve({ data: 'success' })
17     });
18
19     const result = await api.get('/retry-test');
20     expect(result).toEqual({ data: 'success' });
21   });
22 });
```



```
22 });
```

Listing 2.5: 错误处理测试用例

### 2.3.2 WebSocket 服务测试

```
1 describe('WebSocketService', () => {
2   let wsService: WebSocketService;
3
4   beforeEach(() => {
5     wsService = new WebSocketService('ws://localhost:8000');
6   });
7
8   it('应该能够连接到服务器', async () => {
9     await wsService.connect('client-001');
10    expect(wsService.isConnected).toBe(true);
11  });
12
13  it('应该能够发送消息', async () => {
14    await wsService.connect('client-001');
15    wsService.send({
16      type: 'test',
17      data: { message: 'hello' },
18      timestamp: new Date().toISOString()
19    });
20  });
21
22  it('应该能够接收消息', (done) => {
23    wsService.on('test', (message) => {
24      expect(message.data).toEqual({ message: 'hello' });
25      done();
26    });
27  });
28
29  it('连接失败时应该自动重连', async () => {
30    const mockSocket = {
31      connect: vi.fn().mockRejectedValue(new Error('Connection failed'))
32    };
33    // 测试重连逻辑
34  });
35 });
```

Listing 2.6: WebSocket 服务测试用例

### 2.3.3 性能监控测试

```
1 describe('性能监控', () => {
2   let monitor: PerformanceMonitor;
3
4   beforeEach(() => {
5     monitor = new PerformanceMonitor();
6   });
7
8   it('应该能够获取 LCP', () => {
9     const lcpEntry = { startTime: 1500 };
10    mockPerformance.getEntriesByType.mockReturnValueOnce([lcpEntry]);
11    const vitals = monitor.getWebVitals();
12    expect(vitals.lcp).toBe(1500);
13  });
14
15  it('应该检测 LCP 过高', () => {
16    mockPerformance.getEntriesByName.mockReturnValueOnce([{ startTime: 3000
17    }]);
18    const result = monitor.checkPerformanceThresholds();
19    expect(result.passed).toBe(false);
20    expect(result.issues.some(issue => issue.includes('LCP'))).toBe(true);
21  });
22
23  it('应该监控内存使用', () => {
24    mockPerformance.memory.usedJSHeapSize = 50 * 1024 * 1024; // 50MB
25    mockPerformance.memory.totalJSHeapSize = 100 * 1024 * 1024; // 100MB
26    const memoryUsage = monitor.getMemoryUsage();
27    expect(memoryUsage.used).toBe(50 * 1024 * 1024);
28  });
29 });
```

Listing 2.7: 性能监控测试用例

### 2.3.4 安全测试

```
1 describe('安全测试', () => {
2   it('应正确转义用户输入', () => {
3     const userInput = '<script>alert("XSS")</script>';
4     const escapedInput = userInput
5       .replace(/</g, '\&lt;');
6       .replace(/>/g, '\&gt;');
7     expect(escapedInput).not.toContain('<script>');
8   });
9 });
```

```

9
10 it('应验证文件路径, 防止路径遍历攻击', () => {
11     const validPath = '/home/user/data/file.txt';
12     const invalidPath = '../.../etc/passwd';
13     expect(isValidPath(validPath)).toBe(true);
14     expect(isValidPath(invalidPath)).toBe(false);
15 });
16
17 it('应检查CSP策略', () => {
18     const cspHeader = "default-src 'self'; script-src 'self' 'unsafe-inline'";
19     expect(validateCSP(cspHeader)).toBe(true);
20 });
21 });

```

Listing 2.8: 安全测试用例

### 2.3.5 E2E 测试

```

1 test('应该能够处理数据导入流程', async ({ page }) => {
2     await page.goto('http://localhost:5173');
3     const importButton = page.locator('button').filter({ hasText: /导入|
4         import/i });
5     await importButton.click();
6     await expect(page.locator('input[type="file"]')).toBeVisible();
7 });
8
9 test('应该能够响应式布局', async ({ page }) => {
10     await page.goto('http://localhost:5173');
11     await page.setViewportSize({ width: 1920, height: 1080 });
12     await page.setViewportSize({ width: 768, height: 1024 });
13     await page.setViewportSize({ width: 375, height: 667 });
14 });
15
16 test('应该能够完成完整的插值工作流', async ({ page }) => {
17     await page.goto('http://localhost:5173');
18     // 1. 上传数据
19     await page.locator('input[type="file"]').setInputFiles('test-data.geojson');
20     // 2. 配置参数
21     await page.locator('#variogram-model').selectOption('spherical');
22     // 3. 启动插值
23     await page.locator('button#start-kriging').click();
24     // 4. 等待完成

```

```
24   await expect(page.locator('.status-completed')).toBeVisible();  
25 });
```

Listing 2.9: E2E 测试用例

## 2.4 测试执行

### 2.4.1 运行单元测试

```
1 # 运行所有测试  
2 npm test  
3  
4 # 运行测试并监听变化  
5 npm run test:watch  
6  
7 # 运行测试并生成覆盖率报告  
8 npm run test:coverage  
9  
10 # 运行特定测试文件  
11 npx vitest tests/api.test.js  
12  
13 # 运行匹配模式的测试  
14 npx vitest --pattern "api"
```

Listing 2.10: 运行单元测试

### 2.4.2 运行 E2E 测试

```
1 # 运行所有E2E测试  
2 npm run test:e2e  
3  
4 # 运行E2E测试并显示UI  
5 npm run test:e2e:ui  
6  
7 # 运行特定浏览器  
8 npx playwright test --project=chromium  
9  
10 # 调试模式  
11 npx playwright test --debug
```

Listing 2.11: 运行 E2E 测试

# Chapter 3

## 后端测试规范

### 3.1 测试框架

#### 3.1.1 pytest 框架

pytest 是项目使用的主要后端测试框架，支持单元测试、集成测试和异步测试。

##### 3.1.1.1 配置文件

```
1 [pytest]
2 testpaths = .
3 python_files = test_*.py
4 python_classes = Test*
5 python_functions = test_*
6 addopts =
7     -v
8     --tb=short
9     --strict-markers
10    --disable-warnings
11    --cov=app
12    --cov-report=term-missing
13    --cov-report=html
14 asyncio_mode = auto
15 markers =
16     slow: marks tests as slow
17     integration: marks tests as integration tests
18     unit: marks tests as unit tests
```

Listing 3.1: pytest.ini

##### 3.1.1.2 测试依赖

```
1 pytest==7.4.3
2 pytest-asyncio==0.21.1
3 pytest-cov==4.1.0
4 pytest-benchmark==4.0.0
5 requests==2.31.0
6 httpx==0.25.2
```

Listing 3.2: requirements.txt 测试依赖

## 3.2 测试文件结构

### 3.2.1 测试文件分类

后端测试文件共 20 个，按功能分类如下：

#### 3.2.1.1 核心模块测试（12 个文件）

- backend/tests/test\_model\_fusion.py - 模型融合系统测试
- backend/tests/test\_route\_planning.py - 路径规划系统测试
- backend/tests/test\_anomaly\_detector.py - 异常检测器测试
- backend/tests/test\_decision\_threshold\_analyzer.py - 决策阈值分析器测试
- backend/tests/test\_error\_predictor.py - 误差预测器测试
- backend/tests/test\_model\_evaluator.py - 模型评估器测试
- backend/tests/test\_risk\_index\_calculator.py - 风险指数计算器测试
- backend/tests/test\_sampling\_algorithms.py - 采样算法测试
- backend/tests/test\_spatial\_risk\_reporter.py - 空间风险报告器测试
- backend/tests/test\_uncertainty\_classifier.py - 不确定性分类器测试
- backend/tests/test\_websocket.py - WebSocket 服务测试
- backend/test\_backend.py - 后端集成测试

### 3.2.1.2 实时插值系统测试 (5 个文件)

- `realtime_interpolation/tests/test_accuracy.py` - 准确性验证测试
- `realtime_interpolation/tests/test_performance.py` - 性能测试
- `realtime_interpolation/tests/test_integration.py` - 集成测试
- `realtime_interpolation/tests/test_cache_system.py` - 缓存系统测试
- `realtime_interpolation/tests/test_incremental_kriging.py` - 增量克里金测试

### 3.2.1.3 多目标优化系统测试 (1 个文件)

- `multi_objective_optimization/tests/test_core.py` - 核心算法测试

### 3.2.1.4 API 集成测试 (2 个文件)

- `api_test.py` - 基础端点测试
- `api_test_complete.py` - 完整工作流程测试

## 3.3 核心测试用例

### 3.3.1 模型融合测试

```
1 class TestWeightCalculator:
2     """权重计算器测试"""
3
4     @pytest.fixture
5     def calculator(self):
6         return WeightCalculator()
7
8     @pytest.fixture
9     def sample_metrics(self):
10        return [
11            ModelMetrics(
12                model_id="model1",
13                model_name="Model 1",
14                rmse=1.2,
15                mae=0.8,
16                r2=0.85,
17                stability=0.9
```

```

18         ),
19         ModelMetrics(
20             model_id="model2",
21             model_name="Model 2",
22             rmse=1.5,
23             mae=1.0,
24             r2=0.80,
25             stability=0.85
26         ),
27         ModelMetrics(
28             model_id="model3",
29             model_name="Model 3",
30             rmse=1.0,
31             mae=0.7,
32             r2=0.90,
33             stability=0.95
34         )
35     ]
36
37     def test_calculate_weights_rmse_based(self, calculator, sample_metrics)
38     :
39         """测试基于RMSE的权重计算"""
40         weights = calculator.calculate_weights(
41             sample_metrics,
42             method='rmse_based'
43         )
44         assert len(weights) == 3
45         assert all(0 <= w <= 1 for w in weights.values())
46         assert abs(sum(weights.values()) - 1.0) < 1e-6
47
48     def test_calculate_weights_r2_based(self, calculator, sample_metrics):
49         """测试基于R2的权重计算"""
50         weights = calculator.calculate_weights(
51             sample_metrics,
52             method='r2_based'
53         )
54         assert len(weights) == 3
55         assert weights['model3'] > weights['model1'] # model3的R2最高

```

Listing 3.3: 权重计算器测试

### 3.3.2 WebSocket 测试

```
1 @pytest.mark.asyncio
```



```

2 async def test_websocket_connection():
3     """测试 WebSocket 连接"""
4     client = TestClient(app)
5     with client.websocket_connect("/ws/test_client") as websocket:
6         # 发送消息
7         await websocket.send_json({
8             "type": "subscribe_task",
9             "task_id": "task_123"
10        })
11
12        # 接收确认消息
13        response = await websocket.receive_json()
14        assert response["type"] == "subscription_confirmed"
15        assert response["task_id"] == "task_123"
16
17 @pytest.mark.asyncio
18 async def test_task_update_notification():
19     """测试任务更新通知"""
20     client = TestClient(app)
21     with client.websocket_connect("/ws/test_client") as websocket:
22         # 订阅任务
23         await websocket.send_json({
24             "type": "subscribe_task",
25             "task_id": "task_123"
26        })
27         await websocket.receive_json()
28
29         # 触发任务更新
30         await notify_task_update("task_123", {"status": "running"})
31
32         # 接收更新通知
33         update = await websocket.receive_json()
34         assert update["type"] == "task_update"
35         assert update["data"]["status"] == "running"

```

Listing 3.4: WebSocket 异步测试

### 3.3.3 API 测试

```

1 def test_create_kriging_task():
2     """测试创建克里金任务"""
3     response = client.post("/start-kriging", json={
4         "data_id": "data-123",
5         "variogram_model": "spherical",

```

```
6         "range": 100.0,
7         "sill": 1.0,
8         "nugget": 0.1,
9         "grid_resolution": 50
10    })
11    assert response.status_code == 200
12    data = response.json()
13    assert "task_id" in data
14    assert data["status"] == "pending"
15
16 def test_get_task_status():
17     """测试获取任务状态"""
18     # 先创建任务
19     create_response = client.post("/start-kriging", json={...})
20     task_id = create_response.json()["task_id"]
21
22     # 查询状态
23     response = client.get(f"/task-status/{task_id}")
24     assert response.status_code == 200
25     data = response.json()
26     assert "status" in data
27     assert "progress" in data
28
29 def test_get_prediction_result():
30     """测试获取预测结果"""
31     response = client.get(f"/result/prediction/{task_id}")
32     assert response.status_code == 200
33     data = response.json()
34     assert "prediction" in data
35     assert "variance" in data
```

Listing 3.5: API 端点测试

### 3.3.4 性能测试

```
1 def test_incremental_update_performance(benchmark):
2     """测试增量更新性能"""
3     # 准备测试数据
4     subscription = create_test_subscription()
5     new_point = DataPoint(x=50, y=50, value=10, id="test")
6
7     # 基准测试
8     result = benchmark(
9         perform_incremental_update,
```

```
10         subscription,
11         new_point
12     )
13
14     # 性能断言
15     assert result.update_time_ms < 100 # 更新时间小于100ms
16
17 def test_large_dataset_performance(benchmark):
18     """测试大数据集性能"""
19     # 创建大型方差网格 (1000x1000)
20     variance_grid = np.random.rand(1000, 1000)
21
22     # 基准测试
23     result = benchmark(
24         optimize_sampling,
25         variance_grid,
26         n_samples=100
27     )
28
29     # 性能断言
30     assert result.total_time < 10.0 # 总时间小于10秒
```

Listing 3.6: 性能基准测试

## 3.4 测试执行

### 3.4.1 运行单元测试

```
1 # 运行所有测试
2 pytest
3
4 # 运行特定测试文件
5 pytest backend/tests/test_model_fusion.py
6
7 # 运行特定测试函数
8 pytest backend/tests/test_model_fusion.py::TestWeightCalculator::
9     test_calculate_weights_rmse_based
10
11 # 运行标记的测试
12 pytest -m integration
13
14 # 运行测试并生成覆盖率报告
15 pytest --cov=app --cov-report=html
```

```
16  
17 # 详细输出  
18 pytest -v
```

Listing 3.7: 运行后端单元测试

### 3.4.2 运行异步测试

```
1 # 运行异步测试  
2 pytest -m asyncio  
3  
4 # 运行特定异步测试  
5 pytest backend/tests/test_websocket.py::test_websocket_connection
```

Listing 3.8: 运行异步测试

# Chapter 4

## 通讯与集成测试

### 4.1 通讯架构

#### 4.1.1 通讯架构图



### 4.2 WebSocket 通讯测试

#### 4.2.1 连接管理测试

- 测试连接建立和断开
- 测试自动重连机制

- 测试心跳检测
- 测试连接状态管理

#### 4.2.2 消息处理测试

- 测试消息发送和接收
- 测试消息确认机制
- 测试批量消息处理
- 测试消息队列管理

#### 4.2.3 任务订阅测试

- 测试任务订阅和取消订阅
- 测试任务更新通知
- 测试多任务并发订阅
- 测试订阅超时处理

### 4.3 HTTP/REST API 通讯测试

#### 4.3.1 请求处理测试

- 测试 GET 请求缓存
- 测试 POST 请求不缓存
- 测试请求去重
- 测试错误重试机制

#### 4.3.2 响应处理测试

- 测试成功响应处理
- 测试错误响应处理
- 测试超时处理
- 测试响应数据验证

## 4.4 集成测试

### 4.4.1 前后端集成测试

```
1 def test_full_interpolation_workflow():
2     """测试完整的插值工作流"""
3     # 1. 前端上传数据
4     upload_response = client.post("/upload-data", files={
5         "file": ("test.geojson", open("test.geojson", "rb"), "application/
6         json")
7     })
8     data_id = upload_response.json()["data_id"]
9
10    # 2. 启动插值任务
11    task_response = client.post("/start-kriging", json={
12        "data_id": data_id,
13        "variogram_model": "spherical",
14        "range": 100.0,
15        "sill": 1.0,
16        "nugget": 0.1
17    })
18    task_id = task_response.json()["task_id"]
19
20    # 3. 等待任务完成
21    import time
22    for _ in range(60): # 最多等待60秒
23        status_response = client.get(f"/task-status/{task_id}")
24        status = status_response.json()["status"]
25        if status == "completed":
26            break
27        time.sleep(1)
28
29    # 4. 获取结果
30    result_response = client.get(f"/result/prediction/{task_id}")
31    assert result_response.status_code == 200
32    result = result_response.json()
33    assert "prediction" in result
```

Listing 4.1: 前后端集成测试

### 4.4.2 WebSocket 与 HTTP 集成测试

```
1 @pytest.mark.asyncio
```

```
2 async def test_http_task_creation_with_websocket_updates():
3     """测试HTTP创建任务，WebSocket接收更新"""
4     client = TestClient(app)
5
6     # 1. 建立WebSocket连接
7     with client.websocket_connect("/ws/test_client") as websocket:
8         # 2. 订阅任务
9         await websocket.send_json({
10             "type": "subscribe_task",
11             "task_id": "task_123"
12         })
13         await websocket.receive_json() # 确认消息
14
15     # 3. 通过HTTP创建任务
16     task_response = client.post("/start-kriging", json={...})
17     task_id = task_response.json()["task_id"]
18
19     # 4. 通过WebSocket接收更新
20     update = await websocket.receive_json()
21     assert update["type"] == "task_update"
22     assert update["task_id"] == task_id
```

Listing 4.2: WebSocket 与 HTTP 集成测试



# Chapter 5

## 功能模块测试

### 5.1 实时插值系统

#### 5.1.1 功能概述

实时插值系统支持增量更新和实时预测，主要用于处理时序空间数据。

#### 5.1.2 输入输出格式

##### 5.1.2.1 创建订阅请求

```
1 {  
2     "subscription_id": "sub_001",  
3     "data_type": "generic",  
4     "spatial_extent": {  
5         "min_x": 0.0,  
6         "max_x": 100.0,  
7         "min_y": 0.0,  
8         "max_y": 100.0  
9     },  
10    "update_frequency": 5,  
11    "interpolation_params": {  
12        "model_type": "spherical",  
13        "sill": 1.0,  
14        "range": 10.0,  
15        "nugget": 0.1  
16    },  
17    "notification_config": {}  
18 }
```

Listing 5.1: 创建订阅请求

### 5.1.2.2 添加数据点请求

```
1 {
2     "subscription_id": "sub_001",
3     "point": {
4         "id": "point_001",
5         "x": 50.0,
6         "y": 50.0,
7         "value": 10.5,
8         "timestamp": "2026-03-16T10:00:00Z",
9         "metadata": {}
10    }
11 }
```

Listing 5.2: 添加数据点请求

### 5.1.2.3 查询预测响应

```
1 {
2     "x": 50.0,
3     "y": 50.0,
4     "prediction": 10.5,
5     "variance": 0.5,
6     "confidence": 0.95,
7     "version": 1,
8     "timestamp": "2026-03-16T10:00:00Z"
9 }
```

Listing 5.3: 查询预测响应

## 5.1.3 测试用例

### 5.1.3.1 订阅管理测试

```
1 def test_create_subscription():
2     """测试创建订阅"""
3     response = client.post("/subscriptions", json={
4         "subscription_id": "sub_001",
5         "data_type": "sensor",
6         "spatial_extent": {
7             "min_x": 0.0,
8             "max_x": 100.0,
9             "min_y": 0.0,
10            "max_y": 100.0
```

```
11     },
12     "update_frequency": 5
13 })
14 assert response.status_code == 201
15 data = response.json()
16 assert data["subscription_id"] == "sub_001"
17
18 def test_get_subscription():
19     """测试获取订阅信息"""
20     response = client.get("/subscriptions/sub_001")
21     assert response.status_code == 200
22     data = response.json()
23     assert data["subscription_id"] == "sub_001"
24     assert "spatial_extent" in data
25
26 def test_delete_subscription():
27     """测试删除订阅"""
28     response = client.delete("/subscriptions/sub_001")
29     assert response.status_code == 200
30
31     # 验证删除
32     response = client.get("/subscriptions/sub_001")
33     assert response.status_code == 404
```

Listing 5.4: 订阅管理测试

### 5.1.3.2 增量更新测试

```
1 def test_incremental_update():
2     """测试增量更新"""
3     # 1. 创建订阅并添加初始数据
4     client.post("/subscriptions", json={...})
5     client.post("/updates", json={
6         "subscription_id": "sub_001",
7         "point": {"id": "p1", "x": 50, "y": 50, "value": 10}
8     })
9
10    # 2. 添加新数据点
11    response = client.post("/updates", json={
12        "subscription_id": "sub_001",
13        "point": {"id": "p2", "x": 60, "y": 60, "value": 12}
14    })
15    assert response.status_code == 200
16
17    # 3. 查询预测值
```

```

18     query_response = client.get(
19         "/subscriptions/sub_001/query?x=55&y=55"
20     )
21     assert query_response.status_code == 200
22     result = query_response.json()
23     assert "prediction" in result
24     assert "variance" in result

```

Listing 5.5: 增量更新测试

### 5.1.3.3 性能测试

```

1 def test_update_performance(benchmark):
2     """测试更新性能"""
3     subscription = create_test_subscription()
4     new_point = DataPoint(x=50, y=50, value=10, id="test")
5
6     result = benchmark(
7         perform_incremental_update,
8         subscription,
9         new_point
10    )
11
12    assert result.update_time_ms < 100
13    assert result.affected_points > 0

```

Listing 5.6: 性能测试

## 5.2 多目标优化系统

### 5.2.1 功能概述

多目标优化系统使用 NSGA-II 算法，在多个目标（方差、成本、可达性）之间寻找最优解。

### 5.2.2 输入输出格式

#### 5.2.2.1 优化请求

```

1 {
2     "variance_grid": {
3         "shape": [100, 100],
4         "x_coords": [0, 1, 2, ..., 99],

```

```

5     "y_coords": [0, 1, 2, ..., 99],
6     "values": [[0.1, 0.2, ...], ...]
7 },
8     "existing_points": [
9         {"x": 10.5, "y": 20.3},
10        {"x": 15.2, "y": 25.7}
11    ],
12    "n_samples": 20,
13    "weights": {
14        "variance": 0.5,
15        "cost": 0.3,
16        "accessibility": 0.2
17    },
18    "constraints": {
19        "boundary": {
20            "type": "Polygon",
21            "coordinates": [[[0, 0], [100, 0], [100, 100], [0, 100], [0,
22            0]]]
23        },
24        "min_distance": 50,
25        "budget": 10000
26    },
27    "algorithm": "NSGA-II",
28    "algorithm_params": {
29        "population_size": 100,
30        "n_generations": 100,
31        "crossover_prob": 0.9,
32        "mutation_prob": 0.1
33    },
34    "is_async": true
35 }

```

Listing 5.7: 优化请求

### 5.2.2.2 优化结果

```

1 {
2     "success": true,
3     "message": "获取成功",
4     "data": {
5         "task_id": "task_1234567890",
6         "status": "completed",
7         "pareto_solutions": [
8             {
9                 "id": 1,

```

```

10         "objectives": {
11             "variance": 0.125,
12             "cost": 8500,
13             "accessibility": -0.23
14         },
15         "sampling_points": [
16             {"id": 1, "x": 10.5, "y": 20.3, "variance": 0.15},
17             {"id": 2, "x": 15.2, "y": 25.7, "variance": 0.12}
18         ],
19         "metrics": {
20             "total_cost": 8500,
21             "avg_variance": 0.125,
22             "avg_accessibility": 0.23
23         }
24     }
25 ],
26 "recommended_solution": {
27     "id": 5,
28     "objectives": {
29         "variance": 0.135,
30         "cost": 7200,
31         "accessibility": -0.28
32     },
33     "sampling_points": [...],
34     "rank": 1,
35     "crowding_distance": 0.45
36 },
37 "convergence_history": [...],
38 "statistics": {
39     "n_pareto_solutions": 15,
40     "n_evaluations": 10000,
41     "total_time": 8.2,
42     "convergence_generation": 85
43 }
44 }
45 }

```

Listing 5.8: 优化结果

## 5.2.3 测试用例

### 5.2.3.1 优化算法测试

```

1 def test_optimization_algorithm():
2     """测试优化算法"""

```

```

3  # 准备测试数据
4  variance_grid = np.random.rand(50, 50)
5
6  # 创建优化任务
7  response = client.post("/multi-objective/optimize", json={
8      "variance_grid": {
9          "shape": [50, 50],
10         "x_coords": np.arange(50).tolist(),
11         "y_coords": np.arange(50).tolist(),
12         "values": variance_grid.tolist()
13     },
14     "n_samples": 10,
15     "weights": {
16         "variance": 0.5,
17         "cost": 0.3,
18         "accessibility": 0.2
19     }
20 })
21 assert response.status_code == 200
22 task_id = response.json()["data"]["task_id"]
23
24 # 等待完成
25 wait_for_task_completion(task_id)
26
27 # 获取结果
28 result_response = client.get(f"/multi-objective/tasks/{task_id}/results")
29 assert result_response.status_code == 200
30 result = result_response.json()
31
32 # 验证结果
33 assert len(result["data"]["pareto_solutions"]) > 0
34 assert "recommended_solution" in result["data"]

```

Listing 5.9: 优化算法测试

### 5.2.3.2 约束条件测试

```

1 def test_boundary_constraint():
2     """测试边界约束"""
3     response = client.post("/multi-objective/optimize", json={
4         "variance_grid": {...},
5         "n_samples": 10,
6         "constraints": {
7             "boundary": {

```

```

8         "type": "Polygon",
9         "coordinates": [[[0, 0], [100, 0], [100, 100], [0, 100],
10         [0, 0]]]
11     }
12 })
13 # 验证所有采样点都在边界内
14 result = get_optimization_result(response.json()["task_id"])
15 for solution in result["pareto_solutions"]:
16     for point in solution["sampling_points"]:
17         assert 0 <= point["x"] <= 100
18         assert 0 <= point["y"] <= 100
19
20 def test_distance_constraint():
21     """测试距离约束"""
22     response = client.post("/multi-objective/optimize", json={
23         "variance_grid": {...},
24         "n_samples": 10,
25         "constraints": {
26             "min_distance": 50.0
27         }
28     })
29     # 验证采样点之间的最小距离
30     result = get_optimization_result(response.json()["task_id"])
31     for solution in result["pareto_solutions"]:
32         points = solution["sampling_points"]
33         for i in range(len(points)):
34             for j in range(i + 1, len(points)):
35                 distance = calculate_distance(points[i], points[j])
36                 assert distance >= 50.0
37
38 def test_budget_constraint():
39     """测试预算约束"""
40     response = client.post("/multi-objective/optimize", json={
41         "variance_grid": {...},
42         "n_samples": 10,
43         "constraints": {
44             "budget": 10000.0
45         }
46     })
47     # 验证总成本不超过预算
48     result = get_optimization_result(response.json()["task_id"])
49     for solution in result["pareto_solutions"]:

```



```
50      assert solution["metrics"]["total_cost"] <= 10000.0
```

Listing 5.10: 约束条件测试

## 5.3 模型融合系统

### 5.3.1 功能概述

模型融合系统支持多种融合策略，用于整合多个模型的预测结果。

### 5.3.2 输入输出格式

#### 5.3.2.1 创建融合任务请求

```
1 {
2     "models": [
3         {
4             "model_id": "model_1",
5             "model_name": "普通克里金",
6             "predictions": [10.5, 11.2, 10.8],
7             "variances": [0.5, 0.6, 0.4]
8         },
9         {
10            "model_id": "model_2",
11            "model_name": "泛克里金",
12            "predictions": [10.8, 11.0, 10.9],
13            "variances": [0.4, 0.5, 0.3]
14        }
15    ],
16    "config": {
17        "strategy": "weighted_average",
18        "weight_method": "rmse_based",
19        "min_weight": 0.0,
20        "max_weight": 1.0,
21        "normalize": true,
22        "smoothing": false,
23        "enable_cross_validation": true,
24        "enable_stability_check": true,
25        "n_folds": 5
26    },
27    "true_values": [10.6, 11.0, 10.9]
28 }
```

Listing 5.11: 创建融合任务请求

### 5.3.2.2 融合结果

```
1 {
2     "success": true,
3     "result": {
4         "task_id": "fusion_task_001",
5         "fused_predictions": [10.65, 11.1, 10.85],
6         "fused_variances": [0.45, 0.55, 0.35],
7         "weights": {
8             "model_1": 0.5,
9             "model_2": 0.5
10        },
11        "metrics": {
12            "rmse": 0.05,
13            "mae": 0.04,
14            "r2": 0.98
15        },
16        "strategy": "weighted_average",
17        "weight_method": "rmse_based"
18    }
19 }
```

Listing 5.12: 融合结果

### 5.3.3 测试用例

#### 5.3.3.1 权重计算测试

```
1 def test_weight_calculation():
2     """测试权重计算"""
3     response = client.post("/fusion/create-task", json={
4         "models": [
5             {
6                 "model_id": "model_1",
7                 "predictions": [10.5, 11.2, 10.8],
8                 "variances": [0.5, 0.6, 0.4]
9             },
10            {
11                "model_id": "model_2",
12                "predictions": [10.8, 11.0, 10.9],
13                "variances": [0.4, 0.5, 0.3]
14            }
15        ],
16        "config": {
17            "strategy": "weighted_average",
```

```

18         "weight_method": "rmse_based"
19     },
20     "true_values": [10.6, 11.0, 10.9]
21 })
22 assert response.status_code == 200
23 result = response.json()
24
25 # 验证权重
26 weights = result["result"]["weights"]
27 assert abs(sum(weights.values()) - 1.0) < 1e-6
28 assert all(0 <= w <= 1 for w in weights.values())

```

Listing 5.13: 权重计算测试

### 5.3.3.2 融合策略测试

```

1 def test_fusion_strategies():
2     """测试不同融合策略"""
3     strategies = ["weighted_average", "minimum_variance", "ensemble"]
4
5     for strategy in strategies:
6         response = client.post("/fusion/create-task", json={
7             "models": [...],
8             "config": {
9                 "strategy": strategy,
10                "weight_method": "rmse_based"
11            }
12        })
13     assert response.status_code == 200
14     result = response.json()
15     assert result["result"]["strategy"] == strategy

```

Listing 5.14: 融合策略测试

## 5.4 路径规划系统

### 5.4.1 功能概述

路径规划系统用于优化采样点的访问路径，支持多种优化目标和约束条件。

### 5.4.2 输入输出格式

#### 5.4.2.1 路径规划请求

```
1 {
2   "sampling_points": [
3     {
4       "id": "point1",
5       "name": "采样点1",
6       "latitude": 39.9042,
7       "longitude": 116.4074,
8       "priority": 5,
9       "service_time": 10,
10      "is_reachable": true
11    }
12  ],
13  "start_point": {
14    "id": "start",
15    "name": "起点",
16    "latitude": 39.8942,
17    "longitude": 116.3974,
18    "priority": 1,
19    "service_time": 0,
20    "is_reachable": true
21  },
22  "constraints": {
23    "vehicle_type": "car",
24    "time_windows": false,
25    "priority_constraint": false,
26    "max_distance": 100000,
27    "max_duration": 3600,
28    "max_cost": 1000
29  },
30  "optimization_goal": "shortest_distance",
31  "algorithm": "tsp",
32  "return_multiple_routes": true
33 }
```

Listing 5.15: 路径规划请求

#### 5.4.2.2 路径规划结果

```
1 {
2   "success": true,
3   "routes": [
4     {
5       "route_id": "uuid",
6       "point_sequence": ["start", "point1", "point2"],
7       "segments": [
```

```
8         {
9             "from": "start",
10            "to": "point1",
11            "distance": 500,
12            "duration": 60,
13            "cost": 10
14        }
15    ],
16    "total_distance": 5000,
17    "total_duration": 600,
18    "total_cost": 100,
19    "start_time": "2024-01-01T10:00:00",
20    "end_time": "2024-01-01T10:10:00"
21    }
22    ],
23    "best_route": {...},
24    "statistics": {
25        "total_routes": 1,
26        "best_distance": 5000,
27        "computation_time": 1.23
28    },
29    "warnings": []
30 }
```

Listing 5.16: 路径规划结果

### 5.4.3 测试用例

#### 5.4.3.1 路径计算测试

```
1 def test_route_calculation():
2     """测试路径计算"""
3     response = client.post("/api/route-planning/plan", json={
4         "sampling_points": [
5             {
6                 "id": "point1",
7                 "latitude": 39.9042,
8                 "longitude": 116.4074
9             },
10            {
11                "id": "point2",
12                "latitude": 39.9142,
13                "longitude": 116.4174
14            }
15        ],
```

```
16     "start_point": {
17         "id": "start",
18         "latitude": 39.8942,
19         "longitude": 116.3974
20     },
21     "optimization_goal": "shortest_distance",
22     "algorithm": "tsp"
23 })
24 assert response.status_code == 200
25 result = response.json()
26
27 # 验证路径
28 assert len(result["routes"]) > 0
29 assert "best_route" in result
30 assert result["best_route"]["total_distance"] > 0
```

Listing 5.17: 路径计算测试

#### 5.4.3.2 约束验证测试

```
1 def test_constraint_validation():
2     """测试约束验证"""
3     response = client.post("/api/route-planning/plan", json={
4         "sampling_points": [...],
5         "constraints": {
6             "max_distance": 1000,
7             "max_duration": 600
8         }
9     })
10    assert response.status_code == 200
11    result = response.json()
12
13    # 验证约束
14    for route in result["routes"]:
15        assert route["total_distance"] <= 1000
16        assert route["total_duration"] <= 600
```

Listing 5.18: 约束验证测试

## 5.5 其他功能模块

### 5.5.1 采样建议系统

#### 5.5.1.1 功能概述

采样建议系统根据方差栅格生成最优采样点推荐。

#### 5.5.1.2 测试用例

```
1 def test_sampling_recommendations():
2     """测试采样建议生成"""
3     response = client.get(
4         "/api/sampling-recommendations/generate?task_id=task_001&strategy=
5         hybrid&n_recommendations=20"
6     )
7     assert response.status_code == 200
8     result = response.json()
9
10    # 验证结果
11    assert len(result["recommendations"]) == 20
12    assert all("x" in rec and "y" in rec for rec in result["recommendations"])
13    assert all("variance" in rec for rec in result["recommendations"])
```

Listing 5.19: 采样建议测试

### 5.5.2 不确定性分级系统

#### 5.5.2.1 功能概述

不确定性分级系统根据预测方差对空间区域进行分级。

#### 5.5.2.2 测试用例

```
1 def test_uncertainty_classification():
2     """测试不确定性分级"""
3     response = client.post("/uncertainty/classify", json={
4         "task_id": "task-001",
5         "prediction": [[10.5, 11.2, 10.8], [11.0, 10.9, 11.3]],
6         "variance": [[0.5, 0.8, 0.6], [0.7, 0.9, 0.4]],
7         "x_coords": [120.1, 120.2, 120.3],
8         "y_coords": [30.1, 30.2]
9     })
```

```
10     assert response.status_code == 200
11     result = response.json()
12
13     # 验证分级结果
14     assert "statistics" in result
15     assert "color_map" in result
16     assert "critical_zones" in result
```

Listing 5.20: 不确定性分级测试



# Chapter 6

## API 接口测试

### 6.1 API 接口总览

#### 6.1.1 接口分类统计

表 6.1: API 接口分类统计

| 分类     | 接口数量  | 主要功能                |
|--------|-------|---------------------|
| 核心插值   | 8 个   | 插值任务、结果查询、报告生成      |
| 采样优化   | 7 个   | 采样建议、影响评估、预览        |
| 不确定性分析 | 4 个   | 分级、风险指数、决策阈值        |
| 批量处理   | 11 个  | 批量插值、报告、对比          |
| AI 扩展  | 8 个   | 模型评估、误差预测、异常检测、模型融合 |
| 系统管理   | 25 个  | 配置、任务队列、资源监控、性能报告   |
| 专业系统   | 35 个  | 实时插值、多目标优化、路径规划     |
| 数据管理   | 18 个  | 上传、导出、项目管理          |
| 总计     | 116 个 |                     |

### 6.2 核心插值接口

#### 6.2.1 接口列表

| 方法   | 路径                           | 功能        | 文件位置         |
|------|------------------------------|-----------|--------------|
| POST | /start-kriging               | 启动克里金插值任务 | 插值任务接口.py:19 |
| GET  | /task-status/{task_id}       | 查询任务状态    | 任务状态接口.py:17 |
| GET  | /result/prediction/{task_id} | 获取预测结果    | 结果查询接口.py:20 |

| 方法  | 路径                                    | 功能        | 文件位置         |
|-----|---------------------------------------|-----------|--------------|
| GET | /result/variance/{task_id}            | 获取方差结果    | 结果查询接口.py:32 |
| GET | /result/report/{task_id}              | 生成克里金分析报告 | 报告生成接口.py:19 |
| GET | /result/download/{task_id}/{filename} | 下载导出文件    | 结果查询接口.py:46 |
| GET | /progress-detail/{task_id}            | 获取任务进度详情  | 进度详情接口.py:17 |
| GET | /estimated-remaining-time/{task_id}   | 获取预计剩余时间  | 进度详情接口.py:29 |

## 6.2.2 接口测试用例

### 6.2.2.1 启动克里金任务

```

1 def test_start_kriging():
2     """测试启动克里金任务"""
3     response = client.post("/start-kriging", json={
4         "data_id": "data-123",
5         "variogram_model": "spherical",
6         "range": 100.0,
7         "sill": 1.0,
8         "nugget": 0.1,
9         "grid_resolution": 50,
10        "nlags": 6
11    })
12    assert response.status_code == 200
13    data = response.json()
14    assert "task_id" in data
15    assert data["status"] == "pending"
16
17 def test_start_kriging_invalid_parameters():
18     """测试启动克里金任务（无效参数）"""
19     response = client.post("/start-kriging", json={
20         "data_id": "data-123",
21         "variogram_model": "invalid",
22         "range": -100.0,
23         "sill": -1.0
24    })
25    assert response.status_code == 422

```

Listing 6.1: 启动克里金任务测试

#### 6.2.2.2 查询任务状态

```

1 def test_get_task_status():
2     """测试查询任务状态"""

```

```

3     # 先创建任务
4     create_response = client.post("/start-kriging", json={...})
5     task_id = create_response.json()["task_id"]
6
7     # 查询状态
8     response = client.get(f"/task-status/{task_id}")
9     assert response.status_code == 200
10    data = response.json()
11    assert "status" in data
12    assert "progress" in data
13    assert data["task_id"] == task_id
14
15 def test_get_task_status_not_found():
16     """测试查询任务状态（任务不存在）"""
17     response = client.get("/task-status/nonexistent-task")
18     assert response.status_code == 404

```

Listing 6.2: 查询任务状态测试

## 6.3 实时插值系统接口

### 6.3.1 接口列表

| 方法     | 路径                                           | 功能           | 文件  |
|--------|----------------------------------------------|--------------|-----|
| POST   | /subscriptions                               | 创建订阅         | fas |
| GET    | /subscriptions/{subscription_id}             | 获取订阅信息       | fas |
| GET    | /subscriptions                               | 列出所有订阅       | fas |
| DELETE | /subscriptions/{subscription_id}             | 删除订阅         | fas |
| POST   | /updates                                     | 添加数据点并触发增量更新 | fas |
| POST   | /subscriptions/{subscription_id}/data-points | 添加数据点        | fas |
| GET    | /subscriptions/{subscription_id}/query       | 查询指定点的预测值    | fas |
| GET    | /subscriptions/{subscription_id}/prediction  | 获取预测值        | fas |
| GET    | /cache/statistics                            | 获取缓存统计信息     | fas |
| DELETE | /cache                                       | 清除缓存         | fas |
| GET    | /system/status                               | 获取系统状态       | fas |
| GET    | /system/metrics                              | 获取性能指标       | fas |
| GET    | /system/alerts                               | 获取告警列表       | fas |

## 6.3.2 接口测试用例

### 6.3.2.1 创建订阅

```
1 def test_create_subscription():
2     """测试创建订阅"""
3     response = client.post("/subscriptions", json={
4         "subscription_id": "sub_001",
5         "data_type": "sensor",
6         "spatial_extent": {
7             "min_x": 0.0,
8             "max_x": 100.0,
9             "min_y": 0.0,
10            "max_y": 100.0
11        },
12        "update_frequency": 5,
13        "interpolation_params": {
14            "model_type": "spherical",
15            "sill": 1.0,
16            "range": 10.0,
17            "nugget": 0.1
18        }
19    })
20    assert response.status_code == 201
21    data = response.json()
22    assert data["subscription_id"] == "sub_001"
23
24 def test_create_subscription_invalid_extent():
25     """测试创建订阅（无效空间范围）"""
26     response = client.post("/subscriptions", json={
27         "subscription_id": "sub_001",
28         "spatial_extent": {
29             "min_x": 100.0,
30             "max_x": 0.0, # 无效: min > max
31             "min_y": 0.0,
32             "max_y": 100.0
33         }
34     })
35    assert response.status_code == 422
```

Listing 6.3: 创建订阅测试

## 6.4 多目标优化系统接口

### 6.4.1 接口列表

| 方法     | 路径                                              | 功能              | Y |
|--------|-------------------------------------------------|-----------------|---|
| POST   | /multi-objective/optimize                       | 创建并执行优化任务       | f |
| GET    | /multi-objective/tasks/{task_id}/status         | 查询优化任务状态        | f |
| GET    | /multi-objective/tasks/{task_id}                | 获取优化任务信息        | f |
| GET    | /multi-objective/tasks/{task_id}/results        | 获取优化任务结果        | f |
| DELETE | /multi-objective/tasks/{task_id}                | 取消优化任务          | f |
| GET    | /multi-objective/config                         | 获取系统配置          | f |
| GET    | /multi-objective/tasks                          | 获取优化任务列表        | f |
| GET    | /multi-objective/tasks/{task_id}/export/geojson | 导出优化结果为 GeoJSON | f |
| GET    | /multi-objective/status                         | 获取系统状态          | f |

### 6.4.2 接口测试用例

#### 6.4.2.1 创建优化任务

```

1 def test_create_optimization_task():
2     """测试创建优化任务"""
3     variance_grid = np.random.rand(50, 50)
4
5     response = client.post("/multi-objective/optimize", json={
6         "variance_grid": {
7             "shape": [50, 50],
8             "x_coords": np.arange(50).tolist(),
9             "y_coords": np.arange(50).tolist(),
10            "values": variance_grid.tolist()
11        },
12        "n_samples": 10,
13        "weights": {
14            "variance": 0.5,
15            "cost": 0.3,
16            "accessibility": 0.2
17        },
18        "constraints": {
19            "boundary": {
20                "type": "Polygon",
21                "coordinates": [[[0, 0], [50, 0], [50, 50], [0, 50], [0,
0]]]

```

```

22         },
23         "min_distance": 5,
24         "budget": 1000
25     },
26     "algorithm": "NSGA-II",
27     "algorithm_params": {
28         "population_size": 50,
29         "n_generations": 50,
30         "crossover_prob": 0.9,
31         "mutation_prob": 0.1
32     }
33 })
34 assert response.status_code == 200
35 data = response.json()
36 assert "task_id" in data["data"]

```

Listing 6.4: 创建优化任务测试

## 6.5 模型融合接口

### 6.5.1 接口列表

| 方法   | 路径                            | 功能            | 文件位置          |
|------|-------------------------------|---------------|---------------|
| POST | /fusion/create-task           | 创建模型融合任务      | 模型融合接口.py:57  |
| GET  | /fusion/task/{task_id}/status | 获取融合任务状态      | 模型融合接口.py:80  |
| GET  | /fusion/task/{task_id}/result | 获取融合任务结果      | 模型融合接口.py:96  |
| POST | /fusion/compare-strategies    | 比较不同融合策略      | 模型融合接口.py:125 |
| POST | /fusion/optimize-weights      | 优化权重计算方法      | 模型融合接口.py:155 |
| GET  | /fusion/tasks                 | 列出所有融合任务      | 模型融合接口.py:184 |
| GET  | /fusion/strategies            | 列出所有可用的融合策略   | 模型融合接口.py:196 |
| GET  | /fusion/weight-methods        | 列出所有可用的权重计算方法 | 模型融合接口.py:219 |
| GET  | /fusion/status                | 获取模型融合系统状态    | 模型融合接口.py:249 |

### 6.5.2 接口测试用例

#### 6.5.2.1 创建融合任务

```

1 def test_create_fusion_task():
2     """测试创建融合任务"""
3     response = client.post("/fusion/create-task", json={

```

```

4     "models": [
5         {
6             "model_id": "model_1",
7             "model_name": "普通克里金",
8             "predictions": [10.5, 11.2, 10.8],
9             "variances": [0.5, 0.6, 0.4]
10        },
11        {
12            "model_id": "model_2",
13            "model_name": "泛克里金",
14            "predictions": [10.8, 11.0, 10.9],
15            "variances": [0.4, 0.5, 0.3]
16        }
17    ],
18    "config": {
19        "strategy": "weighted_average",
20        "weight_method": "rmse_based"
21    },
22    "true_values": [10.6, 11.0, 10.9]
23 })
24 assert response.status_code == 200
25 data = response.json()
26 assert "task_id" in data["result"]

```

Listing 6.5: 创建融合任务测试

## 6.6 路径规划接口

### 6.6.1 接口列表

| 方法     | 路径                                          | 功能          | 文件 |
|--------|---------------------------------------------|-------------|----|
| POST   | /api/route-planning/plan                    | 执行路径规划      | 路径 |
| POST   | /api/route-planning/optimize                | 优化现有路径      | 路径 |
| POST   | /api/route-planning/add-point               | 向路径中添加新采样点  | 路径 |
| POST   | /api/route-planning/remove-point            | 从路径中移除采样点   | 路径 |
| POST   | /api/route-planning/reorder                 | 重新排序路径中的采样点 | 路径 |
| POST   | /api/route-planning/templates               | 创建路径模板      | 路径 |
| GET    | /api/route-planning/templates               | 获取所有路径模板    | 路径 |
| GET    | /api/route-planning/templates/{template_id} | 获取指定路径模板    | 路径 |
| DELETE | /api/route-planning/templates/{template_id} | 删除路径模板      | 路径 |
| GET    | /api/route-planning/algorithms              | 获取可用的路径规划算法 | 路径 |

| 方法   | 路径                                     | 功能           | 文件 |
|------|----------------------------------------|--------------|----|
| GET  | /api/route-planning/vehicle-types      | 获取支持的车辆类型    | 路径 |
| GET  | /api/route-planning/optimization-goals | 获取支持的优化目标    | 路径 |
| POST | /api/route-planning/validate           | 验证路径是否满足约束条件 | 路径 |

6.7 API 状态码定义

表 6.7: API 状态码定义

| 状态码 | 含义     | 使用场景           |
|-----|--------|----------------|
| 200 | 成功     | 请求成功处理并返回结果    |
| 201 | 创建成功   | 资源创建成功（如任务、项目） |
| 400 | 请求错误   | 参数验证失败、数据格式错误  |
| 401 | 未授权    | 需要认证但未提供       |
| 403 | 禁止访问   | 权限不足           |
| 404 | 未找到    | 资源不存在（任务、数据等）  |
| 409 | 冲突     | 数据冲突、重复提交      |
| 422 | 数据验证失败 | 请求数据不符合验证规则    |
| 429 | 请求过多   | 超出请求频率限制       |
| 500 | 服务器错误  | 服务器内部错误        |
| 502 | 网关错误   | 网关或代理服务器错误     |
| 503 | 服务不可用  | 服务暂时不可用        |
| 504 | 超时     | 请求超时           |



# Chapter 7

## 测试文件与用例

### 7.1 前端测试文件详细列表

#### 7.1.1 API 相关测试文件

表 7.1: API 相关测试文件

| 文件名                                    | 测试内容       | 测试用例数 |
|----------------------------------------|------------|-------|
| tests/api.test.js                      | API 基础功能   | 15    |
| tests/api/api-service.test.ts          | API 服务详细测试 | 25    |
| tests/anomaly-detect-api.test.js       | 异常检测 API   | 10    |
| tests/config-api.test.js               | 配置 API     | 8     |
| tests/decision-thresholds-api.test.js  | 决策阈值 API   | 12    |
| tests/error-predict-api.test.js        | 误差预测 API   | 10    |
| tests/model-evaluation-api.test.js     | 模型评估 API   | 15    |
| tests/risk-calculate-api.test.js       | 风险计算 API   | 10    |
| tests/risk-report-api.test.js          | 风险报告 API   | 8     |
| tests/uncertainty-classify-api.test.js | 不确定性分类 API | 12    |

表 7.2: 功能模块测试文件

| 文件名                              | 测试内容   | 测试用例数 |
|----------------------------------|--------|-------|
| tests/advancedfilter.test.js     | 高级过滤器  | 15    |
| tests/batchoperations.test.js    | 批量操作   | 20    |
| tests/confirmdialog.test.js      | 确认对话框  | 10    |
| tests/datacomparison.test.js     | 数据对比   | 12    |
| tests/exporthenhancer.test.js    | 导出增强   | 15    |
| tests/feedbackcollector.test.js  | 反馈收集器  | 8     |
| tests/formvalidator.test.js      | 表单验证器  | 20    |
| tests/historymanager.test.js     | 历史管理器  | 15    |
| tests/offlinemanager.test.js     | 离线管理器  | 18    |
| tests/preferencespanel.test.js   | 偏好设置面板 | 10    |
| tests/templatedownloader.test.js | 模板下载器  | 8     |
| tests/thememanager.test.js       | 主题管理器  | 12    |

表 7.3: 性能和安全测试文件

| 文件名                               | 测试内容   | 测试用例数 |
|-----------------------------------|--------|-------|
| tests/performance-monitor.test.ts | 性能监控   | 20    |
| tests/performancemonitor.test.js  | 性能监控器  | 15    |
| tests/cache-performance.test.js   | 缓存性能   | 10    |
| tests/security.test.ts            | 安全测试套件 | 25    |
| tests/performance-benchmark.js    | 性能基准测试 | 8     |

### 7.1.2 功能模块测试文件

### 7.1.3 性能和安全测试文件

## 7.2 后端测试文件详细列表

### 7.2.1 核心模块测试文件

### 7.2.2 实时插值系统测试文件

### 7.2.3 多目标优化系统测试文件

## 7.3 测试用例统计

表 7.4: 核心模块测试文件

| 文件名                                               | 测试内容         | 测试用例数 |
|---------------------------------------------------|--------------|-------|
| backend/tests/test_model_fusion.py                | 模型融合系统       | 30    |
| backend/tests/test_route_planning.py              | 路径规划系统       | 25    |
| backend/tests/test_anomaly_detector.py            | 异常检测器        | 15    |
| backend/tests/test_decision_threshold_analyzer.py | 决策阈值分析器      | 12    |
| backend/tests/test_error_predictor.py             | 误差预测器        | 15    |
| backend/tests/test_model_evaluator.py             | 模型评估器        | 20    |
| backend/tests/test_risk_index_calculator.py       | 风险指数计算器      | 12    |
| backend/tests/test_sampling_algorithms.py         | 采样算法         | 18    |
| backend/tests/test_spatial_risk_reporter.py       | 空间风险报告器      | 10    |
| backend/tests/test_uncertainty_classifier.py      | 不确定性分类器      | 15    |
| backend/tests/test_websocket.py                   | WebSocket 服务 | 20    |
| backend/test_backend.py                           | 后端集成测试       | 25    |

表 7.5: 实时插值系统测试文件

| 文件名                                                      | 测试内容  | 测试用例数 |
|----------------------------------------------------------|-------|-------|
| realtime_interpolation/tests/test_accuracy.py            | 准确性验证 | 20    |
| realtime_interpolation/tests/test_performance.py         | 性能测试  | 15    |
| realtime_interpolation/tests/test_integration.py         | 集成测试  | 25    |
| realtime_interpolation/tests/test_cache_system.py        | 缓存系统  | 18    |
| realtime_interpolation/tests/test_incremental_kriging.py | 增量克里金 | 22    |

表 7.6: 多目标优化系统测试文件

| 文件名                                             | 测试内容   | 测试用例数 |
|-------------------------------------------------|--------|-------|
| multi_objective_optimization/tests/test_core.py | 核心算法测试 | 35    |

表 7.7: 测试用例统计

| 类别        | 文件数 | 测试用例数 | 占比   |
|-----------|-----|-------|------|
| 前端单元测试    | 39  | 400   | 45%  |
| 前端 E2E 测试 | 1   | 10    | 1%   |
| 后端单元测试    | 18  | 300   | 34%  |
| 后端集成测试    | 2   | 50    | 6%   |
| 专业系统测试    | 6   | 120   | 14%  |
| 总计        | 66  | 880   | 100% |

# Chapter 8

## 测试执行与报告

### 8.1 测试执行流程

#### 8.1.1 测试准备

1. 安装测试依赖
2. 配置测试环境
3. 准备测试数据
4. 启动测试服务器

#### 8.1.2 运行测试

##### 8.1.2.1 前端测试

```
1 # 1. 安装依赖
2 npm install
3
4 # 2. 运行单元测试
5 npm test
6
7 # 3. 运行E2E测试
8 npm run test:e2e
9
10 # 4. 生成覆盖率报告
11 npm run test:coverage
```

Listing 8.1: 运行前端测试

### 8.1.2.2 后端测试

```
1 # 1. 安装依赖
2 pip install -r requirements.txt
3
4 # 2. 运行单元测试
5 pytest
6
7 # 3. 运行特定模块测试
8 pytest backend/tests/test_model_fusion.py
9
10 # 4. 生成覆盖率报告
11 pytest --cov=app --cov-report=html
```

Listing 8.2: 运行后端测试

### 8.1.2.3 API 集成测试

```
1 # 1. 启动后端服务器
2 python backend/run.py
3
4 # 2. 运行基础API测试
5 python api_test.py
6
7 # 3. 运行完整工作流程测试
8 python api_test_complete.py
```

Listing 8.3: 运行 API 集成测试

## 8.2 测试报告

### 8.2.1 覆盖率报告

#### 8.2.1.1 前端覆盖率报告

前端覆盖率报告生成在 `coverage/` 目录下，包括：

- `coverage/index.html` - HTML 格式报告
- `coverage/coverage-final.json` - JSON 格式报告
- `coverage/lcov.info` - LCOV 格式报告

### 8.2.1.2 后端覆盖率报告

后端覆盖率报告生成在 `htmlcov/` 目录下, 包括:

- `htmlcov/index.html` - HTML 格式报告
- 终端输出显示覆盖率摘要

## 8.2.2 性能报告

### 8.2.2.1 前端性能报告

前端性能测试结果包括:

- Core Web Vitals 指标 (LCP、FID、CLS)
- 内存使用情况
- 资源加载时间
- 性能阈值检查结果

### 8.2.2.2 后端性能报告

后端性能测试结果包括:

- API 响应时间
- 吞吐量 (QPS)
- 资源使用情况 (CPU、内存)
- 算法执行时间

## 8.2.3 测试结果分析

1. 检查测试通过率
2. 分析失败的测试用例
3. 检查覆盖率是否达标
4. 识别性能瓶颈
5. 提出改进建议

## 8.3 持续集成

### 8.3.1 CI/CD 配置

- GitHub Actions 自动运行测试
- 测试失败时阻止合并
- 自动生成测试报告
- 覆盖率阈值检查

### 8.3.2 测试策略

- 每次提交触发单元测试
- 每次 PR 触发完整测试套件
- 每日运行性能测试
- 每周运行 E2E 测试

## 8.4 测试最佳实践

### 8.4.1 编写测试用例

- 遵循 AAA 模式 (Arrange-Act-Assert)
- 测试应该独立且可重复
- 使用有意义的测试名称
- 测试正常情况和边界情况
- 避免测试实现细节

### 8.4.2 Mock 和 Stub

- Mock 外部依赖
- Stub 函数返回值
- 隔离被测单元
- 避免过度 Mock

### 8.4.3 测试数据管理

- 使用测试数据工厂
- 避免硬编码测试数据
- 使用测试数据库
- 清理测试数据

## 8.5 测试维护

### 8.5.1 定期审查

- 每月审查测试覆盖率
- 每季度审查测试策略
- 定期更新测试依赖
- 优化慢速测试

### 8.5.2 问题追踪

- 记录测试失败原因
- 追踪测试问题修复
- 定期 review 测试代码
- 重构重复测试代码



# 附录 A

## 数据格式参考

### A.1 实时插值数据格式

#### A.1.1 DataPoint 数据结构

```
1 @dataclass
2 class DataPoint:
3     """数据点"""
4     x: float          # X坐标
5     y: float          # Y坐标
6     value: float      # 观测值
7     id: str           # 唯一标识符
8     timestamp: Optional[datetime] = None # 时间戳
9     metadata: Optional[Dict[str, Any]] = None # 元数据
```

Listing A.1: DataPoint 数据结构

#### A.1.2 Subscription 数据结构

```
1 @dataclass
2 class Subscription:
3     """订阅"""
4     subscription_id: str
5     data_type: str
6     spatial_extent: BoundingBox
7     update_frequency: int
8     interpolation_params: VariogramModel
9     notification_config: Dict[str, Any]
10    created_at: datetime
```

```
11 updated_at: datetime
```

Listing A.2: Subscription 数据结构

## A.2 多目标优化数据格式

### A.2.1 OptimizationRequest 数据结构

```
1 @dataclass
2 class OptimizationRequest:
3     """优化请求"""
4     variance_grid: Dict[str, Any]
5     existing_points: List[Dict[str, float]]
6     n_samples: int
7     weights: Dict[str, float]
8     constraints: Dict[str, Any]
9     algorithm: str
10    algorithm_params: Dict[str, Any]
11    is_async: bool
```

Listing A.3: OptimizationRequest 数据结构

### A.2.2 ParetoSolution 数据结构

```
1 @dataclass
2 class ParetoSolution:
3     """帕累托解"""
4     id: int
5     objectives: Dict[str, float]
6     sampling_points: List[Dict[str, Any]]
7     metrics: Dict[str, float]
8     rank: int
9     crowding_distance: float
```

Listing A.4: ParetoSolution 数据结构

## A.3 模型融合数据格式

### A.3.1 ModelInput 数据结构

```
1 @dataclass
2 class ModelInput:
3     """模型输入"""
4     model_id: str
5     model_name: str
6     predictions: List[float]
7     variances: List[float]
```

Listing A.5: ModelInput 数据结构

### A.3.2 FusionResult 数据结构

```
1 @dataclass
2 class FusionResult:
3     """融合结果"""
4     task_id: str
5     fused_predictions: List[float]
6     fused_variances: List[float]
7     weights: Dict[str, float]
8     metrics: Dict[str, float]
9     strategy: str
10    weight_method: str
```

Listing A.6: FusionResult 数据结构

# 附录 B

## 参考文档

### B.1 项目文档

- docs/系统架构.md - 系统架构文档
- docs/开发指南.md - 开发指南
- docs/使用手册.md - 用户手册
- docs/插件开发.md - 插件开发文档

### B.2 API 文档

- realtime\_interpolation/接口设计文档.md - 实时插值 API 文档
- multi\_objective\_optimization/API 接口设计文档.md - 多目标优化 API 文档
- docs/插件架构.md - 插件 API 文档

### B.3 算法文档

- realtime\_interpolation/算法设计文档.md - 实时插值算法文档
- multi\_objective\_optimization/算法设计文档.md - 多目标优化算法文档
- docs/模型融合系统文档.md - 模型融合算法文档

### B.4 外部资源

- Vitest 官方文档: <https://vitest.dev/>

- Playwright 官方文档: <https://playwright.dev/>
- pytest 官方文档: <https://docs.pytest.org/>
- FastAPI 官方文档: <https://fastapi.tiangolo.com/>

## 附录 C

### 术语表

**克里金插值** 一种基于统计学的空间插值方法

**NSGA-II** 非支配排序遗传算法 II，用于多目标优化

**帕累托前沿** 多目标优化中的最优解集合

**WebSocket** 一种在单个 TCP 连接上进行全双工通讯的协议

**E2E 测试** 端到端测试，测试完整的用户 workflow

**覆盖率** 测试覆盖的代码比例

**Core Web Vitals** Google 提出的 Web 性能指标

**LCP** Largest Contentful Paint，最大内容绘制时间

**FID** First Input Delay，首次输入延迟

**CLS** Cumulative Layout Shift，累积布局偏移

**API** Application Programming Interface，应用程序接口

**REST** Representational State Transfer，表述性状态转移

**JSON** JavaScript Object Notation，一种轻量级的数据交换格式

**GeoJSON** 一种基于 JSON 的地理空间数据格式

**WebSocket** 一种全双工通信协议

**SSE** Server-Sent Events，服务器推送事件

**Mock** 模拟对象，用于隔离测试

**Stub** 桩函数，用于提供预设的返回值